

metasignal: A Python Package for Comprehensive Metacognitive Analysis and Decision-Making

Saurabh Ranjan¹, Mukesh Makwana², Konstantina Sokratous³, and Brian Odegaard¹

¹ University of Florida, USA ² Brown University, USA ³ University of Missouri, USA

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [✉](#)

Submitted: 31 July 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

metasignal is an open-source Python package for signal detection theory (SDT) and metacognitive measurement. It implements the 17 metacognitive measures evaluated by Rahnev (2025), together with the reference variables d' (perceptual sensitivity), response criterion c (response bias), and mean confidence. The 17 measures comprise three meta- d' -family estimates, *meta- d'* , *M-ratio*, and *M-difference*; four nonparametric Type-2 measures, the Type-2 area under the receiver-operating-characteristic curve (*AUC2*), *Gamma*, *Phi*, and *delta confidence*, together with their eight SDT-normalized ratio and difference forms; and two model-based measures, *meta-noise* and *meta-uncertainty*. A single function computes the complete set from trial-level stimulus, response, and confidence arrays. metasignal currently supports binary (two-alternative) discrimination tasks, in which each trial's stimulus and response are coded with exactly two categories. The package also provides a command-line interface, group summaries, bootstrap confidence intervals, permutation tests, optional hierarchical Bayesian models, and information-theoretic measures. metasignal unifies these measures in a single platform to encourage broader metacognition research and adoption in decision-making studies.

Statement of Need

Metacognition is the ability to evaluate the quality of one's own decisions. It supports learning, adaptive choice, and communication of uncertainty (Fleming & Dolan, 2012; Yeung & Summerfield, 2012). Metacognitive measurement is also relevant to clinical research (Hoven et al., 2019), behavioral economics (Moore & Healy, 2008), and the evaluation of uncertainty reported by artificial-intelligence systems, where metacognitive sensitivity has been proposed as key to calibrating human trust in AI (Griot et al., 2025; D. Lee et al., 2025; Steyvers & Peters, 2025). A consensus statement identified reliable computational models and standardized measurement as major goals for visual-metacognition research (Rahnev et al., 2022).

Numerous metacognitive measures have been proposed, but they quantify distinct properties and vary in their sensitivity to task performance, response bias, confidence bias, and sample size. Among these, meta- d' is widely used because it expresses metacognitive sensitivity in signal detection theory (SDT) units directly comparable to perceptual sensitivity (Maniscalco & Lau, 2012). In plain terms, it measures how well confidence tracks actual accuracy, on the same scale researchers already use for perceptual sensitivity. Although the original MATLAB implementation (Maniscalco & Lau, 2014) and its Python port (A. L. F. Lee, 2018) brought meta- d' to a broad audience, both tools focus exclusively on this single measure. The other sixteen measures benchmarked by Rahnev (2025) remained scattered across isolated codebases, leaving no single Python package that spans the full set. This fragmentation has hindered researchers from comparing metacognitive measures within a single, reproducible workflow.

42 Rahnev (2025) provided the first broad empirical comparison of 17 measures. The study
43 assessed validity and precision, dependence on nuisance variables, split-half reliability, and
44 test-retest reliability. `metasignal` translates this benchmark into a documented Python package
45 that requires no proprietary software. It is intended for cognitive scientists, psychophysicists,
46 neuroscientists, clinical researchers, and researchers studying confidence in artificial systems.

47 State of the Field

48 The MATLAB toolbox by Maniscalco & Lau (2014) and its Python port (A. L. F. Lee, 2018)
49 remain important reference implementations for standard meta- d' estimation. Packages like
50 `metadpy` (Legrand, 2021) and `HMeta-d` (Fleming, 2017) introduced valuable Bayesian and
51 hierarchical estimation frameworks, though they remain focused primarily on the meta- d'
52 family. In R, `statConfR` (Rausch et al., 2025) provides confidence models alongside information-
53 theoretic metrics. While each of these tools excels within its intended scope, none offers the full
54 battery of metrics benchmarked by Rahnev (2025) through a single, native Python interface.

55 `metasignal` addresses this gap by bringing an expansive collection of metacognitive measures
56 under a single Python package umbrella. Rather than replacing existing tools, it complements
57 them by providing broad metric coverage and a standardized trial-level API, which together
58 lower the barrier to wider adoption and more rigorous metacognition research. The optional
59 `sdtbayes` subpackage enables Stan-based hierarchical estimation, and the experimental `itmc`
60 subpackage implements measures grounded in metacognitive information theory (Dayan, 2023;
61 Meyen et al., 2025), with its `statconfr` backend ported from the R `statConfR` package
62 (Rausch et al., 2025).

63 Software Design

64 `metasignal` is built around four primary design principles. First, input data (stimulus identity,
65 behavioral response, and confidence) are structured as tabular trial-level data compatible with
66 standard Python processing workflows. Second, the core `stdpy` layer relies solely on NumPy
67 (Harris et al., 2020) and SciPy (Virtanen et al., 2020), providing a pure-Python implementation.
68 Third, it offers flexible execution: `compute_all_measures` provides a single entry point for
69 broad profiling, while individual functions remain exposed for focused queries. Fourth, complex
70 Bayesian dependencies are isolated from the lightweight frequentist core.

71 When invoked, `compute_all_measures` returns 26 values. The first 20 span d' , decision
72 criterion, mean confidence, and the 17 metacognitive metrics. The final six outputs (log-
73 likelihood, AIC, BIC, AICc, fitted parameter count, and sample size) serve as diagnostic
74 parameters for the meta- d' fit rather than distinct metacognitive measures. An optional
75 `return_type` argument ('dict' or 'dataframe') labels these same 26 values by measure
76 name instead of position; the default 'array' preserves positional, backward-compatible
77 output.

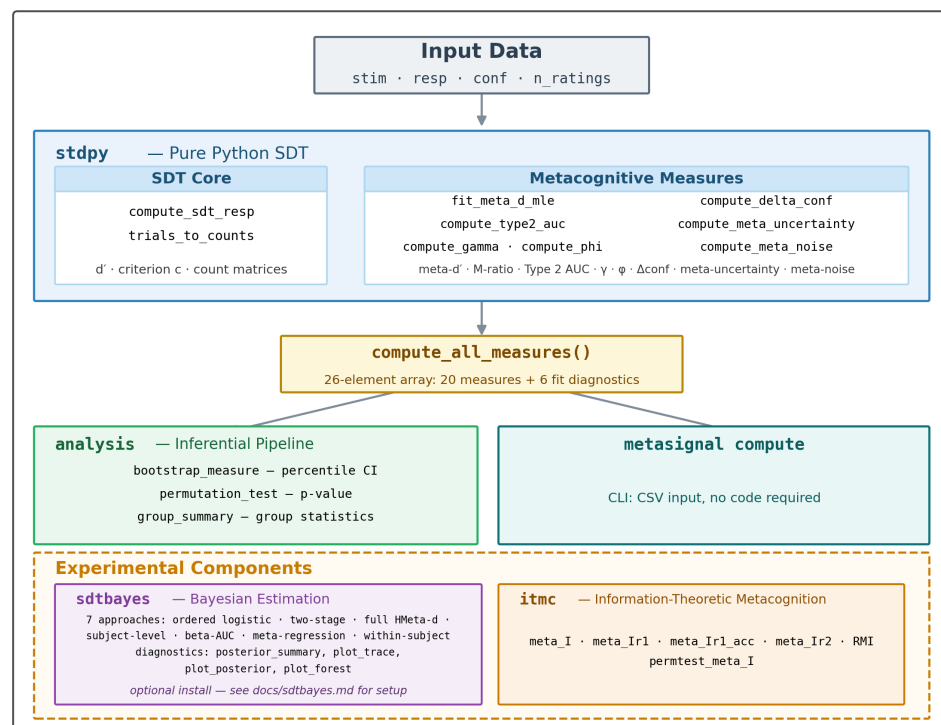


Figure 1: Architecture of metaspinal. Trial-level stimulus, response, confidence, and rating-scale arrays (stim, resp, conf, n_ratings) enter the stable stdpy core, which computes Type-1 SDT quantities (d' , criterion c) and all 17 metacognitive measures; compute_all_measures packages these into a single 26-element array (20 measures plus 6 meta- d' fit diagnostics). This output feeds the analysis layer (bootstrap confidence intervals, permutation tests, group summaries) and the metaspinal compute command-line interface, which accepts CSV input without requiring code. The dashed box shows optional, dependency-isolated extensions: sdtbayes for hierarchical Bayesian estimation (seven modeling approaches) and itmc for information-theoretic metacognition measures.

Methods

Core computation

The package first converts trial-level data into Type-2 response-count arrays. Here, “Type-1” denotes the original stimulus/response decision and “Type-2” the confidence judgment about it, unrelated to Type I/Type II statistical error. Type-1 sensitivity and criterion are computed under the equal-variance SDT model (Green & Swets, 1966). Nonparametric Type-2 measures are computed directly from the observed counts. Meta- d' is estimated by maximum likelihood (Maniscalco & Lau, 2012), from which M-ratio and M-difference are derived. SDT-normalized Ratio and Difference measures compare observed statistics with statistics expected under an ideal equal-variance SDT observer.

Meta-noise is estimated with the lognormal confidence-noise model used by the MATLAB benchmark. The Python implementation follows the MATLAB procedure: it evaluates the zero-noise Gaussian baseline, searches outward from that lower bound, uses golden-section optimization, and evaluates the precomputed integral table using inverse-distance weighting. Meta-uncertainty is fitted with its corresponding model-based estimator.

93 **Validation data and procedure**

94 Validation used the six datasets distributed with the Rahnev analysis pipeline: Haddara,
95 Maniscalco, Rouault experiments 1 and 2, Shekhar, and Locke. The comparison used three
96 sources:

- 97 1. values reported in the text, figures, and supplementary tables of Rahnev (2025);
- 98 2. subject-level MATLAB arrays stored in matlab/metasignal_mat/Results; and
- 99 3. Python arrays generated from the same trial subsets.

100 Ten subject-level analysis arrays were compared: raw estimates, metacognitive-bias recoding,
101 odd-even splits, difficulty conditions, and response-bias conditions. For each measure we
102 examined Pearson correlation, mean absolute error, root-mean-square error, signed bias,
103 maximum absolute error, and missing-value agreement. We also reproduced 19 statistical
104 checks from the supplementary analyses and compared the 17-measure profiles for task-
105 performance dependence, metacognitive-bias dependence, response-bias dependence, and
106 test-retest reliability. The test-retest comparison used the same bin size (400 trials), day
107 structure, and Fisher-z aggregation as the MATLAB scripts. Precision was also recomputed
108 directly from raw trial data using the bin-stratified protocol Rahnev (2025) describes (non-
109 overlapping 50-, 100-, 200-, and 400-trial bins), under a bin-instance cap required for
110 computational tractability (see Limitations).

111 The complete workflow is implemented in analysis/rahnev_comparison/scripts. The JSON
112 results, CSV summaries, identity plots, profile overlays, and combined PDFs are stored beside
113 the scripts. This makes the validation repeatable rather than dependent on a manually prepared
114 table.

Validation Results

metasignal validation: Rahnev (2025) · MATLAB pipeline · Python package

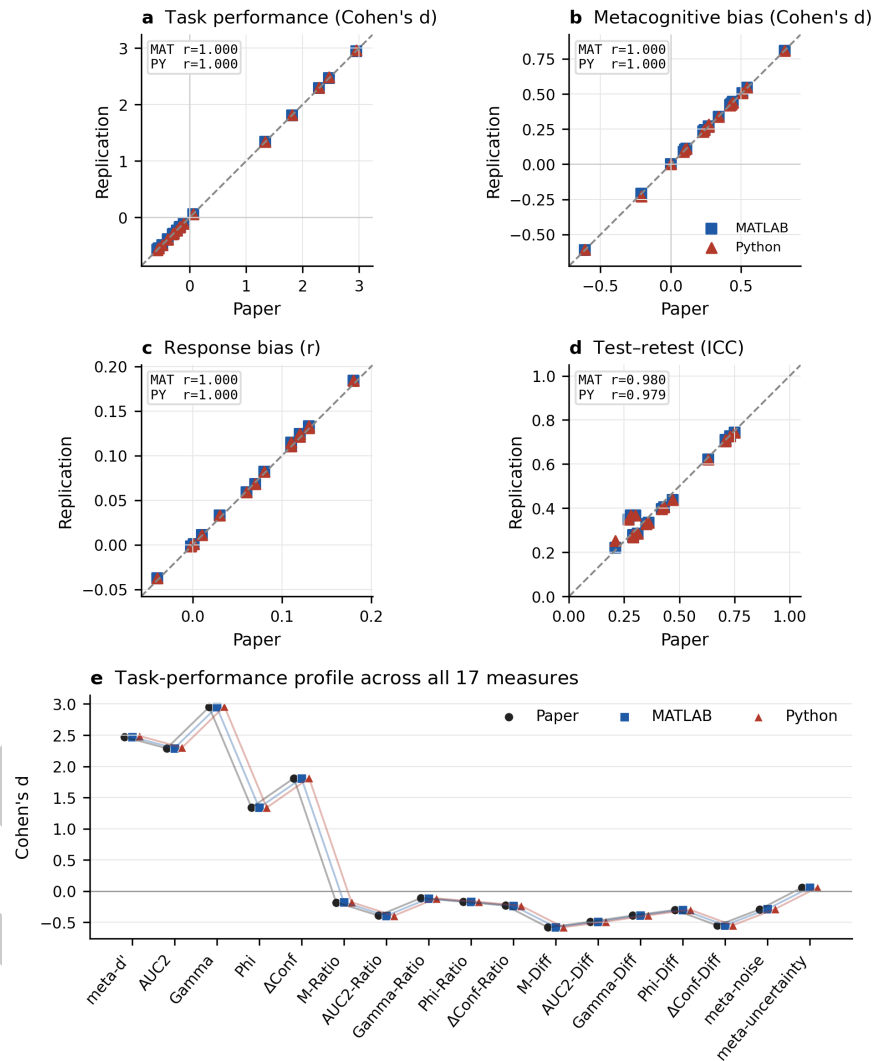


Figure 2: Cross-implementation validation against Rahnev (2025). In each panel, published values (x-axis, "Paper") are plotted against replication values (y-axis) from the MATLAB pipeline (blue squares) and the Python metasignal package (red triangles); the dashed line marks exact identity. Published values in all five panels are the per-measure averages reported in Rahnev (2025)'s Figure 7 ("Summary of results"). MAT and PY report the Pearson correlation between each replication and the published values. **a**, Task-performance effect sizes (Cohen's d ; MAT $r = 1.000$, PY $r = 1.000$). **b**, Metacognitive-bias effect sizes (Cohen's d ; MAT $r = 1.000$, PY $r = 1.000$). **c**, Response-bias correlations (r ; MAT $r = 1.000$, PY $r = 1.000$). **d**, Test-retest reliability (intraclass correlation coefficient, ICC; MAT $r = 0.980$, PY $r = 0.979$). **e**, Task-performance profile (Cohen's d) across all 17 metacognitive measures, comparing published values (black circles) with MATLAB (blue squares) and Python (red triangles) replications.

119 Tables 3, 4, and 6 (Shekhar difficulty, Rouault1 difficulty, and Haddara metacognitive-bias
120 contrasts, respectively; $8 + 4 + 7 = 19$ measure-level values), reproduced independently in both
121 the MATLAB and Python pipelines — distinct from the profile-level correlations reported in
122 the paragraphs below. Full results are in [analysis/rahnev_comparison/ANALYSIS_REPORT.md](#).
123 Across the 18 non-model-based measures, MATLAB and Python agreed to numerical precision
124 at the subject level (maximum systematic bias below 1.5×10^{-3} ; per-analysis correlations of
125 1.000). Ten of these measures agreed to within a maximum absolute difference of 10^{-2} across
126 every subject and analysis.

127 The analysis profiles were essentially identical across sources: task-performance dependence
128 (paper–Python $r = 1.000$; MATLAB–Python $r = 1.000$), metacognitive-bias dependence
129 (paper–Python $r = 1.000$; MATLAB–Python $r = 1.000$), and response-bias dependence
130 (paper–Python $r = 1.000$; MATLAB–Python $r = 1.000$). Test–retest ICC (intraclass
131 correlation coefficient), matched to the MATLAB bin-size-400 protocol, also aligned closely
132 (MATLAB–Python $r = 0.999$).

133 Some differences remain and are reported as limitations rather than hidden. Meta-d', M-ratio,
134 and M-difference correlate at 1.000 across analyses but can differ by up to about 0.14 for
135 individual participants because both pipelines use bounded numerical maximum-likelihood
136 optimizers with different internal solvers. Meta-noise and meta-uncertainty retain a small
137 number of subject-level differences in the sparsest difficulty subsets, where the likelihood surface
138 is flat. A single degenerate value returned by the MATLAB meta-noise search (near 0.4959) is
139 identified programmatically and excluded from agreement statistics. For users who need to
140 reproduce a specific MATLAB result file rather than a scientifically stable estimate, the model-
141 based estimators expose an optional `matlab_compat` mode that mimics MATLAB's single-start
142 optimizer configuration. The package therefore supports the scientific conclusions of Rahnev
143 (2025) and reproduces the MATLAB pipeline to numerical precision for the non-model-based
144 measures, while documenting the residual, well-understood variation in low-information model
145 fits.

146 The experimental `itmc` subpackage was validated separately from the core `stdpy` benchmark
147 above. Its `statconfr` backend reproduces the R `statConfr` package's `estimateMetaI()` output
148 to machine precision on synthetic data spanning a range of sensitivity, bias, and confidence-
149 scale configurations ($r = 1.000$ for all five measures under deterministic computation), and
150 independently reproduces a hand-worked numerical example from Dayan (2023) exactly (meta-I
151 $= 0.094$). Full results, including a known ratio-measure instability at low sensitivity common
152 to both implementations, are in `analysis/itmc_comparison/`.

153 Simple Procedure for Use

154 Installation

155 Install from the public repository:

```
156 pip install git+https://github.com/saurabhr/metasignal.git
```

156 For development or reproducible validation:

```
git clone https://github.com/saurabhr/metasignal.git
cd metasignal
pip install -e .
```

157 Prepare trial-level data

158 Each trial requires four values:

- 159 ▪ `stim`: stimulus category, coded with two values such as 0 and 1;
- 160 ▪ `resp`: participant response, coded with the same two values;

- 161 ▪ conf: integer confidence rating from 1 to n_ratings; and
- 162 ▪ n_ratings: number of possible confidence levels.

163 The arrays must have the same length. Missing trials are removed jointly. A study should
164 compute participant-level estimates separately before group inference.

165 Compute all measures

166 The following self-contained example simulates one participant with a fixed random seed, so
167 that any user can run it and obtain the same numbers. This doubles as a minimal reproducibility
168 check of the installation.

```
import numpy as np
from metasignal import stdpy

rng = np.random.default_rng(2025)
n = 800
stim = rng.integers(0, 2, n)           # two stimulus categories
evidence = (stim * 2 - 1) * 0.9 + rng.normal(0, 1, n)
resp = (evidence > 0).astype(int)      # observer response
edges = np.quantile(np.abs(evidence), [0.25, 0.5, 0.75])
conf = np.digitize(np.abs(evidence), edges) + 1 # 4-point confidence

values = stdpy.compute_all_measures(stim, resp, conf, n_ratings=4)

meta_d = values[0] # meta-d'
auc2 = values[1] # Type-2 AUC
m_ratio = values[5] # M-ratio (meta-d'/d')
dprime = values[17] # d'
print(f"d'={dprime:.3f} meta-d'={meta_d:.3f} M-ratio={m_ratio:.3f} AUC2={auc2:.3f}")
```

169 Running this prints:

```
d'=1.929 meta-d'=1.841 M-ratio=0.955 AUC2=0.762
```

170 compute_all_measures returns all 26 values in a fixed order; the first 20 are the 17
171 metacognitive measures plus d', criterion, and mean confidence, and the last six are meta-d'
172 fit diagnostics. Researchers should use realistic trial counts for scientific estimation; the seed
173 here only makes the example reproducible.

174 Compute selected measures

```
dprime, criterion, ln_beta = stdpy.compute_sdt_resp(stim, resp)
nr_s1, nr_s2 = stdpy.trials_to_counts(stim, resp, conf, n_ratings=4)
meta_fit = stdpy.fit_meta_d_mle(nr_s1, nr_s2)
auc2 = stdpy.compute_type2_auc(nr_s1, nr_s2)

print(meta_fit["meta_da"])
print(meta_fit["M_ratio"])
```

175 Command-line use

```
metasignal compute \
  --stim "0,1,0,1,0,1,0,1" \
  --resp "0,1,0,0,0,1,1,1" \
  --conf "4,4,3,1,3,4,2,3" \
  --n-ratings 4
```

176 Trial data can also be read from a CSV with one trial per row instead of typed inline:

```
cat > trials.csv <<'CSV'
stim,resp,conf
0,0,4
1,1,4
0,0,3
1,0,1
0,0,3
1,1,4
0,1,2
1,1,3
CSV
```

```
metasignal compute --csv trials.csv --n-ratings 4
```

177 Both commands above print the identical 26-value table (they encode the same eight trials);
 178 this was verified directly rather than assumed. Column names default to stim, resp, and conf
 179 and can be overridden with --stim-col, --resp-col, and --conf-col for CSVs with different
 180 headers.

181 Recommended scientific workflow

- 182 1. Check that stimulus and response each contain two categories.
- 183 2. Confirm that confidence values are integers within the declared scale.
- 184 3. Compute measures independently for each participant and condition.
- 185 4. Inspect missing values and convergence diagnostics before group analysis.
- 186 5. Use bootstrap intervals or permutation tests when sampling distributions are uncertain.
- 187 6. Report the measure definition, trial count, confidence scale, exclusion criteria, and
 188 reliability protocol.
- 189 7. For direct replication, preserve the original bin sizes, recoding rules, and random-
 190 resampling procedure.

191 Research Impact Statement

192 metasignal turns a broad methodological benchmark into reusable research infrastructure. It
 193 provides a common implementation for comparing measures, lowers the barrier for laboratories
 194 without MATLAB, and makes numerical validation visible and reproducible. Seven tutorial
 195 notebooks cover preprocessing, measure computation, statistical tables, difficulty dependence,
 196 bias, split-half reliability, and test-retest reliability. Automated tests, continuous integration,
 197 API documentation, and contribution guidelines support reuse and extension.

198 The validation materials are also a scholarly contribution. They identify implementation details
 199 that materially affect results, especially the zero-noise boundary in meta-noise fitting and the
 200 use of proportions in SDT-expected ratings. Recording these details helps prevent apparently
 201 contradictory results that arise from software rather than theory.

202 Limitations

203 Validation Scope

204 No single metacognitive measure is optimal for every experimental design (Rahnev, 2025).
 205 Model-based and ratio measures can become unstable with small sample sizes or sparse
 206 confidence categories, particularly when expected denominators approach zero. Users should
 207 select measures based on their scientific questions and report analysis settings completely.

208 While metasignal perfectly reproduces the core task-performance, metacognitive-bias, and
209 response-bias benchmark profiles from Rahnev (2025) ($r = 1.000$), precision validation requires
210 nuanced interpretation. Precision was recomputed directly from raw trial data using the bin-
211 stratified protocol Rahnev (2025) describes (non-overlapping 50-, 100-, 200-, and 400-trial
212 bins). Under a bin-instance cap required for computational tractability, precision estimates
213 display elevated sampling noise, occasionally causing unstable measures (e.g., Gamma-Ratio)
214 to invert signs. Consequently, current precision figures should be treated as illustrative rather
215 than an exact protocol match to Rahnev (2025).

216 Conclusions

217 metasignal provides a simple, open, and validated Python interface to the principal SDT and
218 metacognitive measures compared by Rahnev (2025). That numerical fidelity means researchers
219 can adopt metasignal in place of the MATLAB pipeline without sacrificing comparability to
220 the published benchmark. The package combines breadth, ease of use, transparent validation,
221 and optional inferential tools, and remaining differences are localized to low-information model
222 fits or unmatched reliability protocols and are documented explicitly. Taken together, these
223 properties make metasignal suitable for reproducible metacognition research while preserving
224 appropriate scientific caution.

225 Code Availability

226 metasignal is open source (MIT license) and available at [https://github.com/saurabhr/](https://github.com/saurabhr/metasignal)
227 [metasignal](https://github.com/saurabhr/metasignal). Full documentation, including the API reference and tutorials, is hosted at
228 <https://metasignal.readthedocs.io/en/latest/>. The complete Rahnev (2025) replication and
229 validation workflow (comparison scripts, JSON/CSV results, and figures) is included in the
230 repository under `analysis/rahnev_comparison/`, and the bundled MATLAB reference pipeline
231 and results are under `matlab/metasignal_mat/`. The original MATLAB code for the 17
232 metacognition measures, from which the bundled MATLAB pipeline is derived, is available at
233 <https://osf.io/y5w2d/> (Rahnev, 2025).

234 AI Usage Disclosure

235 Claude (Anthropic) and OpenAI language models were used to assist with initial drafts of
236 portions of the source code, documentation, validation report, and manuscript. All software
237 behavior, numerical comparisons, citations, scientific claims, and final wording were reviewed
238 and edited by the authors. Responsibility for the software and manuscript remains with the
239 authors.

240 References

- 241 Dayan, P. (2023). Metacognitive information theory. *Open Mind*, 7, 392–411. https://doi.org/10.1162/opmi_a_00091
- 242
- 243 Fleming, S. M. (2017). HMeta-d: Hierarchical bayesian estimation of metacognitive efficiency
244 from confidence ratings. *Neuroscience of Consciousness*, 2017(1), nix007. <https://doi.org/10.1093/nc/nix007>
- 245
- 246 Fleming, S. M., & Dolan, R. J. (2012). The neural basis of metacognitive ability. *Philosophical*
247 *Transactions of the Royal Society B: Biological Sciences*, 367(1594), 1338–1349. <https://doi.org/10.1098/rstb.2011.0417>
- 248
- 249 Green, D. M., & Swets, J. A. (1966). *Signal detection theory and psychophysics*. Wiley.

- 250 Griot, M., Hemptinne, C., Vanderdonckt, J., & Yuksel, D. (2025). Large language models
251 lack essential metacognition for reliable medical reasoning. *Nature Communications*, 16(1),
252 642. <https://doi.org/10.1038/s41467-024-55628-6>
- 253 Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau,
254 D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van
255 Kerkwijk, M. H., Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ...
256 Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.
257 <https://doi.org/10.1038/s41586-020-2649-2>
- 258 Hoven, M., Lebreton, M., Engelmann, J. B., Denys, D., Luigjes, J., & Holst, R. J. van.
259 (2019). Abnormalities of confidence in psychiatry: An overview and future perspectives.
260 *Translational Psychiatry*, 9(1), 268. <https://doi.org/10.1038/s41398-019-0602-7>
- 261 Lee, A. L. F. (2018). *Meta-d' analysis in Python*. [https://sites.google.com/view/alanlflee-](https://sites.google.com/view/alanlflee-lab/resources)
262 [lab/resources](https://sites.google.com/view/alanlflee-lab/resources)
- 263 Lee, D., Pruitt, J., Zhou, T., Du, J., & Odegaard, B. (2025). Metacognitive sensitivity: The
264 key to calibrating trust and optimal decision making with AI. *PNAS Nexus*, 4(5), pgaf133.
265 <https://doi.org/10.1093/pnasnexus/pgaf133>
- 266 Legrand, N. (2021). *Metadpy: Bayesian meta-d' for python*. [https://github.com/LegrandNico/](https://github.com/LegrandNico/metadpy)
267 [metadpy](https://github.com/LegrandNico/metadpy).
- 268 Maniscalco, B., & Lau, H. (2012). A signal detection theoretic approach for estimating
269 metacognitive sensitivity from confidence ratings. *Consciousness and Cognition*, 21(1),
270 422–430. <https://doi.org/10.1016/j.concog.2011.09.021>
- 271 Maniscalco, B., & Lau, H. (2014). *A signal detection theoretic approach for estimating*
272 *metacognitive sensitivity from confidence ratings*. [http://www.columbia.edu/~bsm2105/](http://www.columbia.edu/~bsm2105/type2sdt/)
273 [type2sdt/](http://www.columbia.edu/~bsm2105/type2sdt/).
- 274 Meyen, S., Göppert, F., Schrenk, C., Luxburg, U. von, & Franz, V. H. (2025). Information-
275 theoretic measures of metacognition: Bounds and relation to group performance. *Open*
276 *Mind*, 9, 1728–1762. <https://doi.org/10.1162/OPMI.a.40>
- 277 Moore, D. A., & Healy, P. J. (2008). The trouble with overconfidence. *Psychological Review*,
278 115(2), 502–517. <https://doi.org/10.1037/0033-295X.115.2.502>
- 279 Rahnev, D. (2025). A comprehensive assessment of current methods for measuring
280 metacognition. *Nature Communications*, 16(1), 701. [https://doi.org/10.1038/s41467-025-](https://doi.org/10.1038/s41467-025-56117-0)
281 [56117-0](https://doi.org/10.1038/s41467-025-56117-0)
- 282 Rahnev, D., Balsdon, T., Charles, L., de Gardelle, V., Denison, R., Desender, K., Faivre, N.,
283 Filevich, E., Fleming, S. M., Jehee, J., Lau, H., Lee, A. L. F., Locke, S. M., Mamassian,
284 P., Odegaard, B., Peters, M., Reyes, G., Rouault, M., Sackur, J., ... Zylberberg, A. (2022).
285 Consensus goals in the field of visual metacognition. *Perspectives on Psychological Science*,
286 17(6), 1746–1765. <https://doi.org/10.1177/17456916221075615>
- 287 Rausch, M., Meyen, S., & Hellmann, S. (2025). statConfR: An R package for static models of
288 decision confidence and metacognition. *Journal of Open Source Software*, 10(106), 6966.
289 <https://doi.org/10.21105/joss.06966>
- 290 Steyvers, M., & Peters, M. A. K. (2025). Metacognition and uncertainty communication
291 in humans and large language models. *Current Directions in Psychological Science*.
292 <https://doi.org/10.1177/09637214251391158>
- 293 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
294 Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M.,
295 Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ...
296 SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing
297 in Python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>

298 Yeung, N., & Summerfield, C. (2012). Metacognition in human decision-making: Confidence
299 and error monitoring. *Philosophical Transactions of the Royal Society B: Biological Sciences*,
300 367(1594), 1310–1321. <https://doi.org/10.1098/rstb.2011.0416>

DRAFT